Personal Fitness Application

Python skeleton program - fitness_working.py

```
#Skeleton Program For Oxford AQA International GCSE Computer Science Paper 1
#Developed using Visual Studio
#To be pre-released to centres
#Also available in C# and Visual Basic
# Fitness application working copy

#The text file "activity.txt" needs to be downloaded from the secure key materials area
#for this program to run as required during the exam.

ACTIVITY_FILE = "activity.txt"

class PersonData():
  def __init__(self, ID, Name, Gender, Age, Weight):
    self.ID  = ID
    self.Name = Name
    self.Gender = Gender
    self.Age = Age
    self.Weight = Weight

class Exercise():
  def __init__(self):
    self.PersonID = ""
    self.Age = 0
    self.DayNo = 0
    self.WeekNo = 0
    self.Time = ""
    self.Type = ""
    self.Duration = 0
    self.Distance = 0.0
    self.AverageHeartRate = 0
    self.Weight = 0.0
    self.CaloriesBurned = 0.0

def DisplayUserData(Individual):
  print()
  print("Individual Data")
  print()
  print("    ID: {:>10s}".format(Individual.ID))
  print("  Name: {:>10s}".format(Individual.Name))
  print("   Age: {:>10d}".format(Individual.Age))
  print("Gender: {:>10s}".format(Individual.Gender))
  print("Weight: {:>10.2f} kg".format(Individual.Weight))
  print()

```

Personal Fitness Application

Python skeleton program - fitness_working.py

```
def DisplayNewActivity(Activity, Individual):
  Activity.CaloriesBurned  = CalculateCalories(Activity,Individual)
  print()
  print("New Activity Entered")
  print()
  print("               ID: {:>10s}".format(Activity.PersonID))
  print("              Age: {:>10d}".format(Activity.Age))
  print("           Weight: {:>10.2f} kg".format(Activity.Weight))
  print("  Type of Exercise: {:>10s}".format(Activity.Type))
  print("        Day Number: {:>10d}".format(Activity.DayNo))
  print("       Week Number: {:>10d}".format(Activity.WeekNo))
  print("              Time: {:>10s}".format(Activity.Time))
  print("          Duration: {:>10d} Minutes".format(Activity.Duration))
  print("          Distance: {:>10.2f} km".format(Activity.Distance))
  print("Average Heart Rate: {:>10.2f} Beats Per Minute".format(Activity.AverageHeartRate))
  print("   Calories Burned: {:>10.2f}".format(Activity.CaloriesBurned))
  print()
  return Activity

def EnterName():
  Name = input("Enter person's name: ")
  while len(Name) < 1:
    print("The name entered is not valid.")
    print("A name must be entered.")
    Name = input("Enter person's name: ")
  return Name

def EnterID():
  ID = input("Enter ID (4 digit string): ")
  while len(ID) != 4 or not ID.isnumeric():
    print("The ID entered is not valid")
    print("The ID must be a 4 digit string.")
    ID = input("Enter ID (4 digit string): ")
  return ID

def EnterAge():
  Age = int(input("Enter age (between 10 and 100): "))
  while Age < 10 or Age > 100:
    print("The age entered is not valid.")
    print("Age must be between 10 and 100.")
    Age = int(input("Enter age (between 10 and 100): "))
  return Age

def EnterGender():
```

```
  Gender = input("Enter gender (F or M): ")
  while Gender != 'F' and Gender != 'M':
    print("The gender entered is not valid.")
    print("Gender must be either F or M.")
    Gender = input("Enter gender (F or M): ")
  return Gender

def EnterWeight():
  Weight = float(input("Enter your current weight (kg): "))
  while Weight <= 0:
    print("The weight entered is not valid.")
    Weight = float(input("Enter your current weight (kg): "))
  return Weight

def EnterUserData(Individual):
  print()
  print("User Information")
  print()
  Individual.Name = EnterName()
  Individual.ID = EnterID()
  Individual.Age = EnterAge()
  Individual.Gender = EnterGender()
  Individual.Weight = EnterWeight()
  print()
  return Individual

def CalculateCalories(Activity, Individual):
  if Individual.Gender == 'F':
    CaloriesBurned = Activity.Age * 0.07 - Activity.Weight * 0.06 + Activity.AverageHeartRate * 0.45 - 20
    CaloriesBurned = CaloriesBurned * Activity.Duration / 4
  elif Individual.Gender == 'M':
    CaloriesBurned = Activity.Age * 0.20 + Activity.Weight * 0.09 + Activity.AverageHeartRate * 0.63 - 55
    CaloriesBurned = CaloriesBurned * Activity.Duration / 4
  return CaloriesBurned

def LoadActivity(Individual, ActivityLog):
  ActivityLog = []
  DataFile = open(ACTIVITY_FILE, 'r')
  DataLine = DataFile.readline()
  while DataLine !="":
    Activity = Exercise()
    ActivityData = DataLine.split(",")
    Activity.PersonID = ActivityData[0]
    Activity.Age = int(ActivityData[1])
```

Personal Fitness Application

Python skeleton program - fitness_working.py

```
    Activity.DayNo = int(ActivityData[2])
    Activity.WeekNo = int(ActivityData[3])
    Activity.Time = ActivityData[4]
    Activity.Type = ActivityData[5]
    Activity.Duration = int(ActivityData[6])
    Activity.Distance = float(ActivityData[7])
    Activity.AverageHeartRate = int(ActivityData[8])
    Activity.Weight = float(ActivityData[9])
    Activity.CaloriesBurned = CalculateCalories(Activity,Individual)
    if Activity.PersonID == Individual.ID:
      ActivityLog.append(Activity)
    DataLine = DataFile.readline()
  DataFile.close()
  return ActivityLog

def EnterDayNumber():
  DayNo = int(input("Enter the day number of the Activity (1 to 7): "))
  while DayNo < 1 or DayNo > 7:
    print("Invalid day number.")
    DayNo = int(input("Enter the day number of the Activity (1 to 7): "))
  return DayNo

def EnterWeekNumber():
  WeekNo = int(input("Enter the week number you did the Activity (1 to 52): "))
  while WeekNo < 1 or WeekNo > 52:
    print("Invalid week number.")
    WeekNo = int(input("Enter the week number you did the Activity (1 to 52): "))
  return WeekNo

def EnterTime():
  Valid = False
  while not Valid:
    Valid = True
    Time = input("Enter time of activity (hh:mm): ")
    if len(Time) != 5:
      Valid = False
    elif Time[2] != ":":
      Valid = False
    elif not Time[0:2].isdigit():
      Valid = False
    elif not Time[3:5].isdigit():
      Valid = False
    if not Valid:
      print("Time is not in a valid format.")
```

Personal Fitness Application

Python skeleton program - fitness_working.py

```
  return Time

def EnterDuration():
  Duration = int(input("Activity Duration (in minutes): "))
  while Duration <= 0:
    print("Duration of activity must be greater than 0")
    Duration = int(input("Activity Duration (in minutes): "))
  return Duration

def EnterDistance():
  Distance = float(input("Activity Distance (in km): "))
  while Distance <= 0.0:
    print("Distance must be greater than 0.0")
    Distance = float(input("Activity Distance (in km): "))
  return Distance

def EnterHeartRate():
  AverageHeartRate = int(input("Enter average heart rate during activity (Beats per minute): "))
  while AverageHeartRate <= 0:
    print("Average heart rate must be greater than 0")
    AverageHeartRate = int(input("Enter average heart rate during activity (Beats per minute): "))
  return AverageHeartRate

def AddActivity(Individual, ActivityLog):
  Activity = Exercise()
  Activity.PersonID = Individual.ID
  Activity.Age = Individual.Age
  Activity.Weight = Individual.Weight
  print()
  print("Add an Activity")
  print()
  print("ID: ", Activity.PersonID)
  print()
  Activity.DayNo = EnterDayNumber()
  Activity.WeekNo = EnterWeekNumber()
  Activity.Time = EnterTime()
  Activity.AverageHeartRate = EnterHeartRate()
  Activity.Type = input("Type of Activity (e.g. running, swimming, cycling): ")
  Activity.Duration = EnterDuration()
  Activity.Distance = EnterDistance()
  Activity = DisplayNewActivity(Activity, Individual)
  ActivityLog.append(Activity)
  AppendActivityToFile(Activity)
  return ActivityLog
```

```
221
222 def SortByDistance(ActivityLog):
223   N = len(ActivityLog)
224   for Pass in range(N - 1):
225     for Index in range(N - 1):
226       if ActivityLog[Index].Distance < ActivityLog[Index + 1].Distance:
227         Temp = ActivityLog[Index]
228         ActivityLog[Index] = ActivityLog[Index + 1]
229         ActivityLog[Index + 1] = Temp
230   return ActivityLog
231
232 def DisplayActivity(ActivityLog, TypeOfActivity, Individual):
233   print()
234   print("ID: ",Individual.ID)
235   print()
236   print("{:>6s}{:>8s}{:>5s}{:>10s}".format("Day No", "Week No", "Time", "Activity"), end="")
237   print("{:>9s}{:>9s}{:>16s}{:>17s}".format("Duration", "Distance", "Avg Heart Rate", "Calories Burned"))
238   print("{:>6s}{:>8s}{:>7s}{:>9s}{:>7s}{:>6s}{:>13s}".format(" ", " ", " ", " ", "(Minutes)", "(km)", "(BPM)"))
239   for Activity in ActivityLog:
240     if TypeOfActivity.lower() == Activity.Type.lower():
241       print("{:>3d}{:>8d}{:>8s}{:>9s}".format(Activity.DayNo, Activity.WeekNo, Activity.Time, Activity.Type), end="")
242       print("{:>7d}{:>10.2f}{:>12d}{:>18.2f}".format(Activity.Duration, Activity.Distance, Activity.AverageHeartRate, Activity.CaloriesBurned))
243   print()
244
245 def AppendActivityToFile(Activity):
246   DataLine = "\n" + Activity.PersonID
247   DataLine = DataLine + "," + str(Activity.Age)
248   DataLine = DataLine + "," + str(Activity.DayNo)
249   DataLine = DataLine + "," + str(Activity.WeekNo)
250   DataLine = DataLine + "," + Activity.Time
251   DataLine = DataLine + "," + Activity.Type
252   DataLine = DataLine + "," + str(Activity.Duration)
253   DataLine = DataLine + "," + str(Activity.Distance)
254   DataLine = DataLine + "," + str(Activity.AverageHeartRate)
255   DataLine = DataLine + "," + str(Activity.Weight)
256   DataFile = open(ACTIVITY_FILE, 'a')
257   DataFile.write(DataLine)
258   DataFile.close()
259
260 def DisplayMenu():
261   print()
262   print("MAIN MENU")
263   print()
264   print("1. Display User Information")
```

Personal Fitness Application

Python skeleton program - fitness_working.py

```
  print("2. Load Activity Log File")
  print("3. Add Activity")
  print("4. Display Activity Details")
  print("5. Sort Activities by Distance")
  print("Q. Quit")
  print()

def GetChoice():
  Choice = input("Please enter your choice: ")
  print()
  return Choice

def Main():
  Individual = PersonData("1001", "NameOne", "M", 21, 70.2)
  Choice = input("Enter D to use the default person data: ")
  if Choice != 'D':
    Individual = EnterUserData(Individual)
  ActivityLog= []
  MenuOption = '0'
  while MenuOption != 'Q':
    DisplayMenu()
    MenuOption = GetChoice()
    if MenuOption == '1':
      DisplayUserData(Individual)
    elif MenuOption == '2':
      ActivityLog = LoadActivity(Individual, ActivityLog)
    elif MenuOption == '3':
      ActivityLog = AddActivity(Individual, ActivityLog)
    elif MenuOption == '4':
      TypeOfActivity = input("What type of activity do you want to display: ")
      DisplayActivity(ActivityLog,TypeOfActivity, Individual)
    elif MenuOption == '5':
      ActivityLog = SortByDistance(ActivityLog)
    elif MenuOption == 'Q':
      print("Goodbye, thank you for using this application.")

if __name__ == "__main__":
  Main()
```